応用情報技術者試験　テクノロジ系　#13

# ソフトウェア開発管理技術

用語集まとめ ― 過去問10回分（令和3〜7年度）とシラバスの用語例から



# 全体像 シラバスの4つの項目を、そのまま4章で読む

**1 開発プロセス・手法**
開発モデル・アジャイル宣言・XP・スクラム・見積りとふりかえり・リーン・DevOpsとFour Keys・ローコード・部品・マッシュアップ・形式手法・SLCP・プロセス能力水準

**2 知的財産適用管理**
著作権と著作者人格権・職務著作・外注の権利・特許と実施権・ライセンスとOSS・DRM

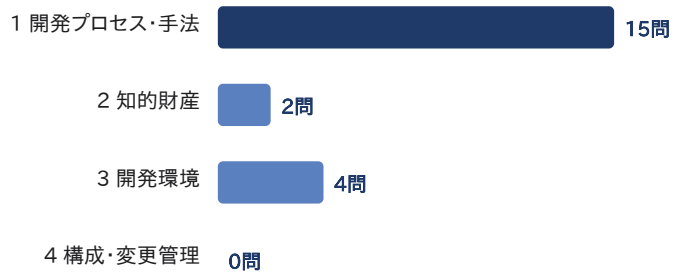
**3 開発環境管理**
開発・テスト・ステージング・本番の環境・IDEと組込み開発の道具・ドキュメンテーションジェネレーター・サーバプロビジョニング・ライセンスの棚卸

**4 構成管理・変更管理**
構成品目・ベースライン・変更管理の手順・完全性の保証・バージョン管理・SBOM

| 午前での出方 | |
|---|---|
| 問題数 | 毎回 **2問前後**（問48〜問50あたり）。10回で21問 |
| 中心 | **アジャイル・スクラムが15問**。役割・イベントの順・ベロシティ・KPTなど |
| 同じ問題の再出題 | IDEの説明（R4春50 と R5秋50） |
| 隣の分野 | テストやリファクタリング「継承の分割」は 12、見積りやEVMは 14 プロジェクトマネジメント |

**この資料の読み方：**スクラムは「誰が・いつ・何を」を表で覚える。似た名前（継続的デリバリーとデプロイ、専用と通常の実施権）は一言の違いで覚える。

# 7割がアジャイル。スクラムの役割とイベントは毎年のように出る




令和3年度春期〜令和7年度秋期の21問を、章ごとに数えた（同じ問題も1問ずつ数えた）

| よく出る型 | 中身 | 過去問 |
|---|---|---|
| スクラムの役割 | 優先順位はプロダクトオーナー、支援はスクラムマスター | R3春49・R3秋50 |
| スクラムのイベント | 順番と、ステークホルダに見せるのはどれか | R5春48・R7春50 |
| 計画と見積り | ベロシティ・バーンダウンチャート・インセプションデッキ | R4春49・R3秋49・R6秋49 |
| XPのプラクティス | ペアプログラミング・テスト駆動開発・KPT | R3春50・R7春48・R4秋49・R4秋50 |
| 考え方と規格 | アジャイル宣言・バリューストリームマップ・JIS X 33002 | R5秋49・R6春48・R6春49 |
| 道具と権利 | IDE・ドキュメンテーションジェネレーター・プロビジョニング・特許・著作者人格権 | R4春50・R6春50・R5春50・R5春49・R7秋50 |

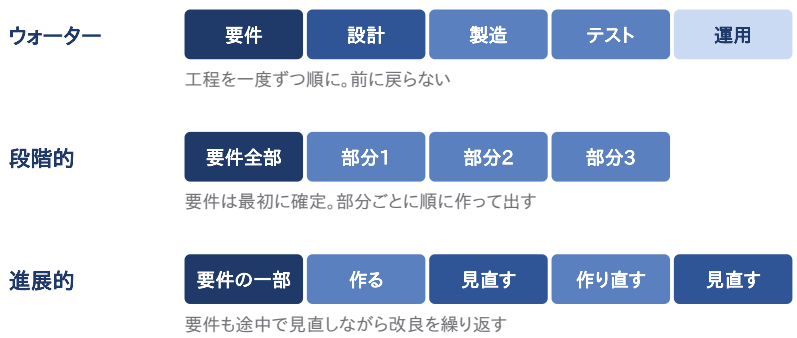
第1章

# 開発プロセス・手法

開発モデル ／ アジャイル宣言 ／ XP ／ スクラム ／ 見積りとふりかえり ／ リーン ／ DevOpsとFour Keys ／ ローコードと部品 ／ マッシュアップとモバイル ／ 形式手法 ／ SLCPとプロセス能力水準



# 開発モデルは「要件をいつ決めるか」と「何を繰り返すか」で見分ける




| モデル | 見分けるキーワード |
| --- | --- |
| ウォーターフォール | 工程を順に一度ずつ。手戻りが大きい |
| プロトタイピング | 試作品を見せて要件を固める |
| スパイラル | 毎回**リスク分析**をしながら渦巻き状に繰り返す |
| **段階的（インクリメンタル）** | 要件は**最初に確定**。部分ごとに順に提供 |
| **進展的（エボリューショナル）** | 要件も**途中で見直す**。作っては改良 |
| ソフトウェアプロダクトライン | 似た製品群の共通部分（コア資産）を再利用 |

> アジャイルは、短い期間（イテレーション）で動くものを作り、変化に合わせて進める考え方。期間は**タイムボックス**で固定し、延ばさない。

# 📜 アジャイルソフトウェア開発宣言：右にも価値はあるが、左により価値をおく

| より価値をおく | | 価値があると認める |
|---|---|---|
| 個人と対話 | よりも | プロセスやツール |
| 動くソフトウェア | よりも | 包括的なドキュメント |
| 顧客との協調 | よりも | 契約交渉 |
| 変化への対応 | よりも | 計画に従うこと |

## R5秋 問49：「別のことがら（より価値をおく方）」の組

正解は**ア**「個人と対話・動くソフトウェア・顧客との協調・変化への対応」。
左右を混ぜた組と、右だけの組（エ）が誤りの選択肢。

「ドキュメントや計画は不要」とは言っていない。右側にも**価値があると認めた上で**、左側をより重く見る。

| 12の原則から | 中身 |
|---|---|
| 顧客の満足 | 価値のあるソフトウェアを早く継続的に届ける |
| 変更を歓迎 | 開発の後半でも要求の変更を受け入れる |
| 進み具合の尺度 | 動くソフトウェアこそが進み具合を表す |
| 持続可能なペース | 一定のペースを保ち続けられるように |
| ふりかえり | 定期的にやり方を見直して調整する |

# アジャイルの手法：XPは「やり方」、スクラムは「チームの枠組み」

| 手法 | 特徴 |
| --- | --- |
| XP | ペアプログラミング・テスト駆動開発など**開発の具体的なやり方**を決める。五つの価値 |
| スクラム | 役割・イベント・作成物を決めた**チームの進め方の枠組み**。プログラミングのやり方は決めない |
| リーンソフトウェア開発 | トヨタ生産方式。**ムダをなくし**、価値を速く届ける |
| FDD | 利用者に価値のある**機能（フィーチャー）**を単位に、設計と実装を繰り返す |
| カンバン | 作業をカードで見える化し、作業中の数に上限（WIP制限） |

大規模にするとき：SAFe（企業全体・階層ごとの役割）、LeSS（スクラムのまま複数チーム、バックログは一つ）、Scrum of Scrums（チームの代表の打合せ）、DA（状況に合わせてやり方を選ぶ）。

**XPの五つの価値**

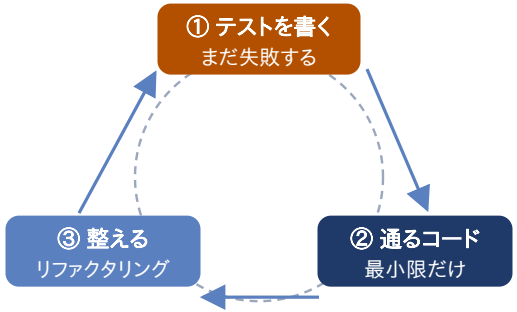
- コミュニケーション
- シンプル
- フィードバック
- 勇気
- 尊重

**XPのプラクティスの4分類**

| | |
| --- | --- |
| **共同** | 反復・共通の用語・開けた作業空間・回顧 |
| **開発** | テスト駆動開発・ペアプログラミング・リファクタリング・ソースコードの共同所有・継続的インテグレーション・YAGNI |
| **管理者** | 責任の受入れ・援護・四半期ごとの見直し・ミラー・最適なペース |
| **顧客** | ストーリーの作成・リリース計画・受入れテスト・短期リリース |

# テスト駆動開発は「先にテスト」、ペアプログラミングは「2人で1つ」

## R4秋 問49：テスト駆動開発の特徴




正解は**エ**「プログラムを書く**前に**テストコードを記述する」。「バグを多く見つける」「カバレージを高める」は目的ではない。

振る舞い駆動開発（BDD）は、テストの前に「～のとき、～すると、～となる」の形で振る舞いを言葉で合意する、TDDの発展形。

## R3春 問50：ペアプログラミングの説明

| | 選択肢の中身 | 本当は |
|---|---|---|
| ア | プログラマと**利用者**が試作の画面を見て相談 | プロトタイピング |
| イ | メインとサブを**分担**して作る | 分担は一緒に作ることにならない |
| ウ | 交互に作業と**休憩**で長時間 | 交代勤務 |
| **エ** | **2人で相談・レビューしながら一つのプログラム** | **正解**。品質向上と知識共有 |

R7春 問48 も「XPのプラクティスはどれか」でペアプログラミングが正解。構造化・コンポーネント指向・ビジュアルプログラミングは誤り。3人以上なら**モブプログラミング**。

# スクラムの3つの役割：「何を」はPO、「どう作る」は開発者、「支える」はSM

**プロダクトオーナー（1人）**
プロダクトの価値を最大にする。プロダクトバックログの項目と**優先順位を決める**。リリースの判断もする

**スクラムマスター（1人）**
スクラムの理論とプラクティスを全員が理解し実践できるよう**支援する**。障害を取り除く。命令はしない

**開発者（数人）**
**どう作るかを自分たちで決め**、スプリントごとに使える状態のインクリメントを作る

スクラムチームは合わせて通常10人以下。上下関係はない。チームの外で関わる利用者・顧客・経営者などが**ステークホルダ**。

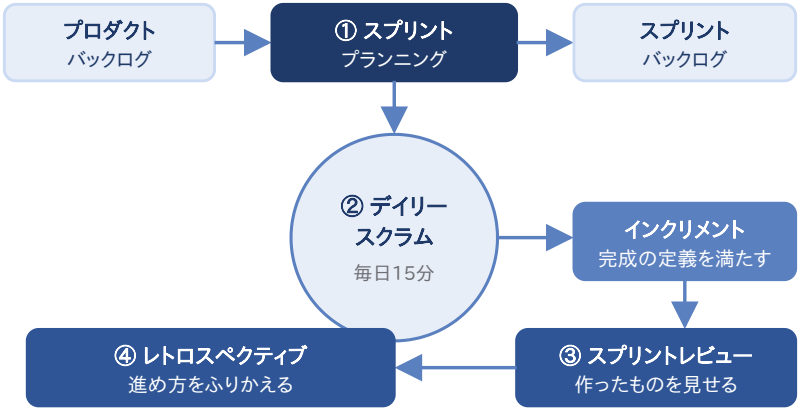
**R3春 問49：プロダクトオーナーの役割**

| | 選択肢の中身 | 誰の役割か |
|---|---|---|
| ア | **プロダクトバックログの優先順位を決定** | **PO（正解）** |
| イ | コーチやファシリテータとして支援 | スクラムマスター |
| ウ | 具体的な作り方を決定 | 開発者 |
| エ | インクリメントを完成する | 開発者 |

**R3秋 問50：スクラムマスターが行うこと**

正解は**イ**「スクラムの理論とプラクティスを全員が理解するように支援」。リリースの可否・バックログの明確化・優先順位の決定は、どれも**プロダクトオーナー**。

# スプリントの中のイベントは「計画 → 毎日 → 見せる → ふりかえる」




1か月以内のスプリントを繰り返す。期間は延ばさない（タイムボックス）

**R5春 問48：イベントの順序**

1 スプリントプランニング → 4 デイリースクラム → 3 スプリントレビュー → 2 スプリントレトロスペクティブ。正解は**イ（1→4→3→2）**。

**R7春 問50：ステークホルダに結果を見せ、スコープを調整するイベント**

| イベント | 見直すもの・すること |
|---|---|
| スプリントプランニング | 今回作るものと作り方を計画 |
| デイリースクラム | 開発者がゴールへの進み具合を毎日確認 |
| **スプリントレビュー** | **作ったもの**をステークホルダに見せ、バックログを調整（正解イ） |
| レトロスペクティブ | **進め方**（人・プロセス・道具）を見直す |

「レビュー＝プロダクト」「レトロスペクティブ＝プロセス」。この順で、レビューが先。

# 作成物と見積り：ストーリーをポイントで相対的に見積もり、ベロシティで測る

| 作成物 | 中身 | 管理 |
|---|---|---|
| プロダクトバックログ | プロダクト全体のやることを優先順に並べた一覧 | PO |
| スプリントバックログ | 今回のスプリントで作る項目と作業の計画 | 開発者 |
| インクリメント | 完成の定義を満たした、使える成果物 | 開発者 |

目標は2段：**プロダクトゴール**（長期、一つだけ）と**スプリントゴール**（今回）。**完成の定義**を満たさない項目はバックログに戻す。

| ストーリー | 中身 |
|---|---|
| ユーザーストーリー | 「〈利用者〉として、〈目的〉のために、〈機能〉がほしい」 |
| INVEST | 独立・交渉可能・価値・見積り可能・小さい・テスト可能 |
| エピック | 1スプリントで作れない大きなストーリー |

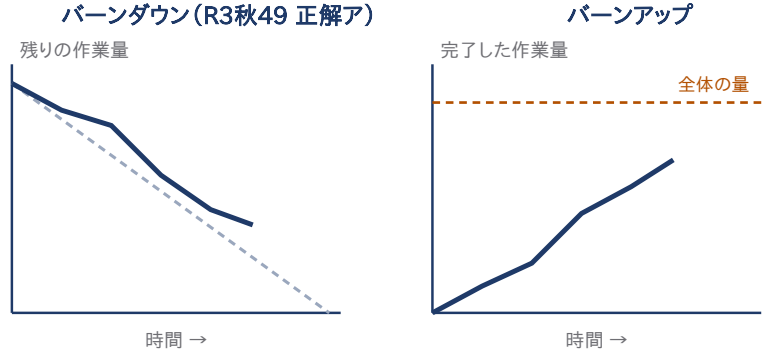
| 見積り | 中身 |
|---|---|
| ストーリーポイント | 手間の大きさを**相対的な数**（1・2・3・5・8…）で |
| プランニングポーカー | 全員が見積りカードを**一斉に**出し、ずれたら理由を話して出し直す |
| **ベロシティ** | 1スプリントでこなしたポイントの合計。**チームの生産量の相対的な尺度** |

**R4春 問49：生産量を相対的に表す尺度**

正解は**エ ベロシティ**。スプリント（期間）・スプリントレトロスペクティブ（イベント）・バックログ（一覧）は尺度ではない。

ベロシティは**チームどうしで比べない**。ポイントの大きさの感覚はチームごとに違うから。

# バーンダウンチャートは「残り」が右下がり。破線が予定、実線が実績




実線が破線より**上**にあれば、予定より遅れている。バーンアップは、途中で全体の量が増えたことも見える。

| R3秋 問49：誤りの選択肢のグラフ | | |
|---|---|---|
| | **縦軸と形** | **本当は** |
| **ア** | **残作業量・右下がり** | **バーンダウンチャート** |
| イ | 発生不具合数・バスタブ型 | 故障率のバスタブ曲線 |
| ウ | 累積バグ数・S字 | 信頼度成長曲線（12で学んだゴンペルツ曲線） |
| エ | 要員数・階段状 | 要員の投入計画 |

S字のバグ曲線は「テストの収束」を見るもの。アジャイルの進み具合はバーンダウン。

# 初めに目的をそろえる：インセプションデッキ

**R6秋 問49：初期段階に、決められた設問で共通認識を作る手法**

| | 選択肢 | 本当は |
|---|---|---|
| ア | アジャイルモデリング | 必要な分だけ簡単に図を描いて共有 |
| **イ** | **インセプションデッキ** | **正解。**10の問いに関係者で答え合う |
| ウ | プランニングポーカー | ストーリーポイントの見積り |
| エ | ユーザーストーリーマッピング | 行動の流れに沿ってストーリーを並べる |

インセプションデッキの問いの例：「我々はなぜここにいるのか」「エレベーターピッチ」「**やらないことリスト**」「夜も眠れない問題」「何がどれだけ必要か」。

**ユーザーストーリーマッピング**




一列に並べたバックログでは見えない**全体の流れ**が分かり、線（図の破線）を引いて、その上を最初のリリースの範囲にできる。利用者像は**ペルソナ**で具体的に描く。

# 🤔 ふりかえりのKPT：続けること・問題・次に試すこと

**Keep**
うまくいったので続ける

**Problem**
うまくいかなかったこと・困ったこと

**Try**
次の期間に新しく試す具体的な行動。Problem の解決や Keep をさらによくする策

スプリントレトロスペクティブで使う。Problem を出すだけで終わらせず、**次にやることを Try で決める**のが要。

**R4秋 問50：「T」に当たる例**

| | 例 | 分類 |
|---|---|---|
| ア | 欠陥が少なかったので、今後もペアプログラミングを続ける | Keep |
| イ | スタンドアップミーティングに全員が集まらないことが多かった | Problem |
| **ウ** | **次のスプリントから、タイムキーパーを置き終了5分前を知らせる** | **Try（正解）** |
| エ | テストコードの作成に見積り以上の時間が掛かった | Problem |

「～した」「～だった」は事実（Keep か Problem）。「次から～する」が Try。

# リーンはムダをなくす。バリューストリームマップで「待ち」を見つける

**R6春 問48：プロセスと所要時間を可視化し、ボトルネックや無駄を確認する道具**



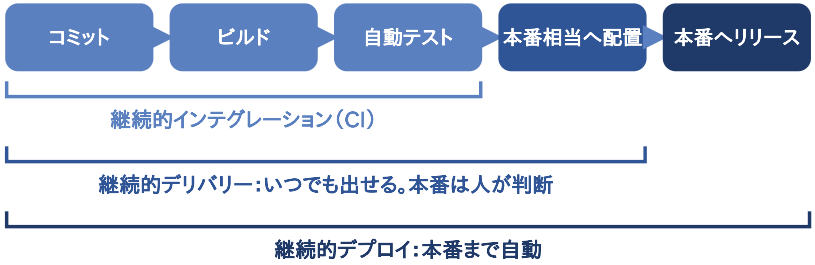

**作業 8日 ／ 全体 24日 → 待ちが16日。ムダはプロセスの「間」にある**

正解は**エ バリューストリームマップ**。ストーリーカード・スプリントバックログ・バーンダウンチャートはプロセスの所要時間を表さない。

| リーンソフトウェア開発の七つの原則 | |
|---|---|
| 1 | ムダをなくす |
| 2 | 品質を作り込む |
| 3 | 知識を作り出す |
| 4 | 決定を遅らせる（確かな情報がそろうまで） |
| 5 | 速く提供する |
| 6 | 人を尊重する |
| 7 | 全体を最適化する |

カンバンで**作業中の数に上限（WIP制限）**をかけると、詰まった所が目に見える。**MVP**（最小限の製品）を早く出して反応を確かめるのもリーンの考え方。

# DevOps：自動化でこまめに届ける。どこまで自動かで名前が変わる




**継続的デリバリー**と**継続的デプロイ**の違いは「本番へのリリースを人が判断するか」だけ。どちらもCDと略すので注意。

| 用語 | 中身 |
|---|---|
| CALMS | 文化・自動化・リーン・測定・共有。道具だけではDevOpsにならない |
| SRE | 運用をソフトウェア工学で。信頼性の目標（SLO）を決める |
| エラーバジェット | SLO 99.9% なら、許される失敗は 0.1%。使い切りそうなら新機能を止める |
| DevSecOps | セキュリティの確認を最初から流れに組み込む |
| MLOps | 機械学習のモデルの学習・配置・監視・再学習を回し続ける |
| オブザーバビリティ | ログ・メトリクス・トレースから内部の状態を理解できる度合い。標準が OpenTelemetry |
| カオスエンジニアリング | 本番で小さく障害を起こして弱さを見つける（12で学んだ） |

# 🔑 Four Keys：速さの2つと、安定性の2つ

| | 指標 | よい状態 |
|---|---|---|
| 速さ | デプロイの頻度 | 本番へのリリースが多い |
| | 変更のリードタイム | コミットから本番まで短い |
| 安定性 | 変更障害率 | デプロイが障害を起こす割合が低い |
| | 平均修復時間（MTTR） | 障害から戻るまでが短い |

GoogleのDORAの研究から生まれた。速さと安定性は対立せず、よいチームは**両方とも**よい、という結果が出ている。

**オリジナル問題：安定性を表す組**

正解は「**変更障害率と平均修復時間**」。デプロイの頻度・変更のリードタイムは速さ。速さと安定性を一つずつ混ぜた組が誤りの選択肢になる。

MTTRは 03・04 の稼働率の計算でも出てくる（平均修理時間）。Four Keysでは「障害からサービスが戻るまで」の時間。

# 🧩 作らずに組み立てる：ローコードと部品の再利用

| R5秋 問47：GUIで部品を組み合わせ、必要なら一部だけコードを書く手法 | | |
|---|---|---|
| | **選択肢** | **本当は** |
| ア | 継続的インテグレーション | 変更をこまめに統合して自動でビルドとテスト |
| イ | ノーコード開発 | コードを**一切書かない** |
| ウ | プロトタイピング | 試作品で要件を固める |
| **エ** | **ローコード開発** | **正解**。一部だけ書く |

速く作れるが、ツールの機能を超えにくく、特定の製品に縛られやすい。部署で勝手に作られて管理されないアプリにも注意。

| 部品の種類（対になる組で覚える） | |
|---|---|
| 関数部品 ／ オブジェクト部品 | 呼び出して使う ／ クラスを継承して使う |
| データ部品 ／ プロセス部品 | 共通のデータ定義 ／ 業務の処理手順 |
| 常駐部品 ／ 組込み部品 | メモリで共有して呼ぶ ／ プログラムに取り込む |
| ブラックボックス ／ ホワイトボックス | 中を見ずに使う ／ ソースを見て直して使う |
| パラメトリック ／ ノンパラメトリック | パラメータで動きを変えられる ／ 変えられない |
| クローズド ／ オープンシステム部品 | 特定の環境だけ ／ 標準に従いどこでも |

部品は**再利用される前提**で作る：モジュールの独立性を高く、カスタマイズはパラメータや継承で、ライブラリで管理し、命名規則で探しやすく。

# APIを組み合わせるマッシュアップと、モバイルアプリの3つの作り方

| マッシュアップ | 組み合わせるもの |
|---|---|
| プレゼンテーション | 画面の表示部品（地図・天気）を一つの画面に並べる |
| データ | 複数のサービスのデータを合わせて新しい情報に |
| ロジック | 予約・決済・通知などの処理をつないで新しい処理に |

早く作れるが、元のAPIの変更や停止の影響を受ける。品質は提供元に依存する。

**リバースエンジニアリング**で既存のソフトを解析して互換製品を作るときも、権利者の許可がなければ知的財産権の侵害になり得る。使用許諾契約で禁止されていることもある。呼出し関係の図は**コールグラフ**。

| | Webアプリ | ハイブリッド | ネイティブ |
|---|---|---|---|
| 作り方 | HTML・JavaScript | Webの技術＋アプリの外枠 | OSごとの言語 |
| 入手 | URLで開く | ストアから | ストアから（審査あり） |
| 端末の機能 | 限られる | 使える | 全部使える |
| OSごとの開発 | 不要 | ほぼ不要 | 必要 |

**PWA**はWebアプリだが、サービスワーカーでオフライン起動やホーム画面への追加ができる。ストアを通さずに配れる。

モバイルの留意点：端末の画面の大きさの違い、圏外や着信で中断されたときの対応、カメラなどを使う前の**パーミッション要求**。

# 形式手法：数学に基づく言語で、あいまいさのない仕様を書いて確かめる

| 用語 | 中身 |
|---|---|
| **形式手法** | 形式仕様記述言語で厳密に書き、正しさを検証する。**モデルの状態**の記述に重点 |
| VDM-SL | VDMの仕様記述言語。型と、操作の前後で成り立つ条件（事前条件・事後条件）を書く |
| VDM++ | VDM-SLをオブジェクト指向に拡張 |
| VDMTools | VDMの仕様の型を調べたり、動かして確かめたりする道具 |
| Z言語 | 集合論と述語論理で、状態と操作をスキーマで書く |
| モデル検査 | 起こり得る**全部の状態**を機械的に調べる（デッドロックがないか など） |
| SPIN | 並行システムのモデル検査の道具 |

テストは一部の場合を試すだけだが、モデル検査は**全部の状態**を網羅する。鉄道の制御・宇宙・ICカードなど、誤りが許されない所で使われる。

| 構造化手法 | 中身 |
|---|---|
| 段階的詳細化 | 大まかな機能から順に細かく分ける |
| 階層構造化 | 分けた結果を上下の階層で表す |
| 構造化チャート | モジュールの呼出し関係の木 |
| HIPO・DFD・状態遷移図 | 12で学んだ図法 |

「形式手法」と「構造化手法」の取り違えに注意。**厳密な言語**なら形式手法、**上から分ける**なら構造化手法。

# SLCP：ソフトウェアの一生の作業を、プロセスのグループで整理する

**合意プロセス**
取得（買う側）と供給（売る側）の合意

**組織のプロジェクトイネーブリングプロセス**
組織としてプロジェクトを支える：ライフサイクルモデル管理・人的資源管理・品質管理など

**テクニカルマネジメントプロセス**
プロジェクトを管理する：計画・評価と管理・意思決定・**リスク管理**・**構成管理**・情報管理・測定・**品質保証**

**テクニカルプロセス**
実際に作り動かす：要件定義・アーキテクチャ・設計・実装・統合・検証・移行・妥当性確認・運用・保守・廃棄

| 規格・枠組み | 対象 |
|---|---|
| JIS X 0160 | ソフトウェアライフサイクルプロセス（ISO/IEC/IEEE 12207） |
| JIS X 0170 | システムライフサイクルプロセス（ISO/IEC/IEEE 15288） |
| 共通フレーム | SLCPを日本向けにまとめた、発注者と受注者の**共通の言葉**（SLCP-JCF） |

**プロセス** > **アクティビティ** > **タスク**

**プロセス修整（テーラリング）**：規格のプロセスをプロジェクトに合わせて取捨選択する。全部満たせば**完全適合**、修整した範囲で満たせば**修整適合**。

共通フレームは作業の内容と用語を決めたもの。開発の方法論や文書の様式までは決めていない。

# プロセス能力は6段階。組織の標準でレベル3、数値の管理でレベル4

**JIS X 33020 のプロセス能力水準**

- **5　革新しているプロセス**　事業の変化に合わせて継続的に改善
- **4　予測可能なプロセス**　測定し、定量的に管理して結果が予測できる
- **3　確立されたプロセス**　組織の標準プロセスを修整して使う
- **2　管理されたプロセス**　計画し、進み具合を監視・調整
- **1　実施されたプロセス**　目的は達成。やり方は人それぞれ
- **0　不完全なプロセス**　実施されていない・目的を達成していない

| 規格・モデル | 中身 |
|---|---|
| JIS X 33001 | プロセスアセスメントの概念と用語 |
| **JIS X 33002** | プロセスアセスメントを**実施する要求事項**。組織のプロセスを客観的に**診断**するため |
| JIS X 33020 | 能力水準 0〜5 の測り方 |
| CMMI | 組織の成熟度レベル1（初期）〜5（最適化） |

**R6春 問49：JIS X 33002 の説明**

正解は**イ**「組織プロセスの品質を客観的に**診断**するための要求事項」。継続的改善の要求事項（品質マネジメント）・プロジェクトマネジメントの手引き・SLCPの共通枠組み（JIS X 0160）は別の規格。

第2章

# 知的財産適用管理

著作権と著作者人格権 ／ 職務著作と外注 ／ 特許と実施権 ／ ライセンスとOSS ／ DRMなどの技術的保護



# © 著作権は「表現」を、特許権は「アイデア」を守る

| | 著作権 | 特許権 | 営業秘密 |
|---|---|---|---|
| 守るもの | プログラム・文書などの**表現** | 技術的な**発明** | 秘密のノウハウ・顧客情報 |
| 手続 | **不要**（作った時点で発生） | 出願・審査・登録 | 不要（3条件を満たす） |
| 期間 | 死後70年（法人は公表後70年） | 出願から**20年** | 秘密である限り |
| 法律 | 著作権法 | 特許法 | 不正競争防止法 |

プログラム言語・規約（プロトコル）・解法（アルゴリズム）は**著作権法では守られない**。アルゴリズムは特許になり得る。

営業秘密の3条件：**秘密管理性**（秘密として管理）・**有用性**（事業に役立つ）・**非公知性**（知られていない）。

**著作権の中身**

**著作者人格権（譲れない）**
公表権 ・ 氏名表示権 ・ 同一性保持権（勝手に変えられない）

**著作財産権（譲れる・許諾できる）**
複製権 ・ 翻案権（作り変え） ・ 公衆送信権 ・ 譲渡権 など

**職務著作**：会社の発意で従業員が職務として作ったプログラムは、契約や就業規則に別の定めがなければ**会社が著作者**。プログラムは会社の名前で公表しなくてもよい。著作者人格権も会社が持つ。

# 🤝 外注したプログラムの著作権は、決めなければ「作った委託先」に残る

**委託元（発注）**
お金を払う

→

**委託先（開発）**
作った＝著作者

| 契約で決めること | 理由 |
|---|---|
| 著作権を委託元へ移す | 決めなければ委託先に残る |
| 翻案権（27条）も含めて移す | 書かないと委託先に残ると推定される |
| **著作者人格権を行使しない** | 人格権は譲れないので、使わないと約束させる |

| R7秋 問50：「委託先は著作者人格権を行使しない」の目的 選択肢の中身 | |
|---|---|
| ア | 開発者が技術者倫理の責任を負わなくなることを防ぐ |
| イ | 委託元が後で開発者を雇用できなくなることを防ぐ |
| ウ | 不具合のとき委託先が金銭的補償を負わなくなることを防ぐ |
| **エ** | **委託先の了解なく委託元で修正できなくなる、他社に修正を頼めなくなることを防ぐ** |

正解は**エ**。委託先に残る**同一性保持権**を使われると、委託元はプログラムを自由に直せない。

# 💡 特許の許諾が要るのは「日本で・業として・20年以内」に実施するとき

| R5春 問49：許諾が必要になる場合 | | |
|---|---|---|
| | **場面** | **許諾** |
| ア | 家庭内で個人的に使うだけ | 不要（業としてではない） |
| **イ** | **日本国内で製造し、全部を国外へ輸出** | **必要（国内の製造が実施）** |
| ウ | 出願日から25年を超えた後に事業化 | 不要（20年で消滅） |
| エ | 出願日より前から独自に開発し、製造販売していた | 不要（先使用権） |

**実施**＝業として、物を生産・使用・譲渡・輸出・輸入すること。作るだけでも実施になる。

| | **専用実施権** | **通常実施権** |
|---|---|---|
| 独占 | **独占**。決めた範囲で一者だけ | 独占でない。何人にも許せる |
| 登録 | 特許庁への**設定登録が必要** | 不要 |
| 特許権者 | その範囲では**自分も実施できない** | 自分も実施できる |

**クロスライセンス**は互いの特許を使い合う契約。**ビジネスモデル特許**は情報システムで実現する事業の方法の特許（アイデアだけでは取れない）。

# ライセンス：許諾する側・受ける側、OSSは条件を守れば自由に使える

**ライセンサー**
許諾する側（権利者）
→
**ライセンシー**
許諾を受けて使う側

| 契約の形 | 中身 |
|---|---|
| 使用許諾契約 | 使える台数・期間・禁止事項（リバースエンジニアリングの禁止など） |
| ボリュームライセンス | 多くの台数分をまとめて許諾 |
| サイトライセンス | 決めた場所の中なら台数を数えない |
| サブスクリプション | 期間ごとに利用料を払って使う |

> ライセンシーは、使用実態や使用人数が契約の範囲を**超えないように管理**する責任がある（3章の棚卸）。

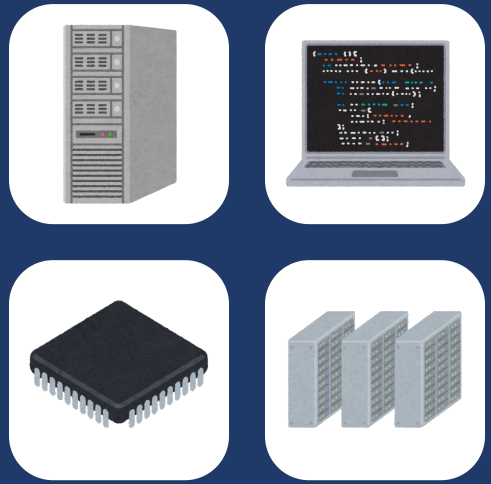
| OSS | 改変して配布するとき |
|---|---|
| GPL | 派生物も**GPLでソースを公開**する（コピーレフト）。販売はしてよい |
| MIT・BSD | 著作権表示とライセンス文を残せば、公開の義務はない |

| 技術的な保護 | |
|---|---|
| DRM | デジタルコンテンツの再生や複製の範囲を制限する仕組みの総称 |
| CPRM | DVD-R・SDカードへの録画の保護（DRMの一種） |
| AACS | ブルーレイディスクの保護（DRMの一種） |
| アクティベーション | インストール後にメーカーの認証を受けないと使えない |
| 電子透かし | 見えない情報を埋め込み、不正コピーの出所を追う |

第3章

# 開発環境管理

開発・テスト・ステージング・本番 ／ IDEと組込み開発の道具 ／ ドキュメンテーションジェネレーター ／ サーバプロビジョニングとIaC ／ ライセンスの棚卸



# 環境を分けて、開発中の変更が本番に響かないようにする




| 道具・仕組み | 中身 |
|---|---|
| シミュレーター | 実機がなくても動きをまねて試す |
| エミュレーター | 別のCPUの命令を再現して、そのまま動かす |
| クロス開発 | パソコン（ホスト）で作り、組込み機器（ターゲット）用に変換 |
| コンテナ | アプリと必要なライブラリを一つにまとめ、どこでも同じに動かす |

| 開発環境の管理 | 中身 |
|---|---|
| 資源管理 | コンピュータ資源の割当てと使用状況 |
| 運用管理 | 起動・停止・バックアップ・障害対応 |
| 更新履歴管理 | 設計データの「いつ・誰が・何を・なぜ」 |
| アクセス権管理 | 誰が読める・変えられるか。機密データは利用の記録も |
| ツール管理 | ツールとバージョンをチームでそろえる |
| **棚卸** | インストール数と保有ライセンス数を**定期的に照合** |

> ツールのバージョンが人によって違うと、作ったソフトウェアの**互換性の問題**や、ツールのバグによる不具合が起きる。ツールとそのバージョンも**構成品目**として管理する。

# 💻 IDEは「道具をひとまとめにしたソフト」。組込み開発のハードの道具と区別する

**R4春 問50 ＝ R5秋 問50：IDEの説明**

| | 選択肢の中身 | 本当は |
|---|---|---|
| **ア** | **エディタ・コンパイラ・リンカ・デバッガなどが一体のツール** | **IDE（正解）** |
| イ | 専用のハードウェアインタフェースでCPUの情報を取得する装置 | ICE |
| ウ | ターゲットCPUを搭載した評価ボードなどの実行環境 | 評価ボード |
| エ | タスクスケジューリングの仕組みを提供するソフトウェア | リアルタイムOS |

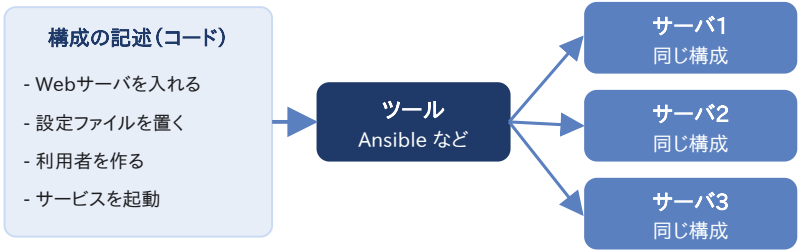
**CASEツール**は開発を支援する道具の総称。上流CASE（要件・設計の図）、下流CASE（プログラミング・テスト。IDEもここ）、保守CASE。

**R6春 問50：ドキュメンテーションジェネレーター**

| | 選択肢の中身 | 本当は |
|---|---|---|
| ア | HTML・CSSを読み込んで画面に描画 | レンダリングエンジン |
| **イ** | **ソースコードのコメントからプログラムのドキュメントを生成** | **正解**（Javadoc など） |
| ウ | テンプレートとデータを合成してWebページを生成 | テンプレートエンジン |
| エ | マーク付けされたテキストを印刷用に組版 | 組版ソフトウェア（TeX） |

どの選択肢も「文書を作る・表示する」ソフト。決め手は**ソースコードのコメントから**。

# プロビジョニングは「書いた構成どおりに自動で組み立てる」




構成を**コード**として書く考え方が **IaC**（Infrastructure as Code）。バージョン管理やレビューができ、同じ環境を何度でも作れる。手作業による設定の違いがなくなる。

**R5春 問50：サーバプロビジョニングツールの目的**

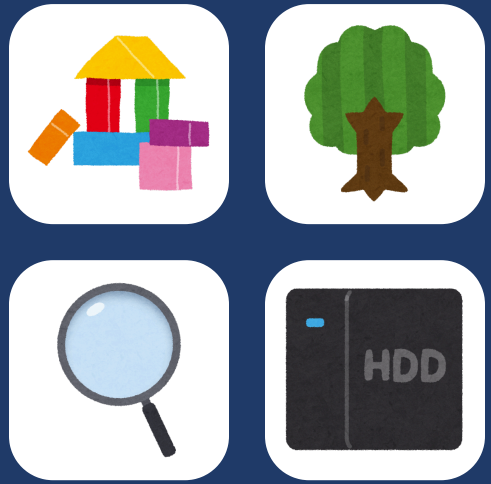
| | 選択肢の中身 | 本当は |
|---|---|---|
| ア | サービスが動いているかを他システムから監視 | 死活監視 |
| イ | インストールされたソフトウェアを一元管理 | ソフトウェア資産管理 |
| ウ | システムやアプリのパフォーマンスを管理 | 性能監視（APM） |
| **エ** | **システム構成を記述しておき、サーバを自動的に構成** | **正解** |

プロビジョニング＝「必要なものを用意して使える状態にする」。見張る（監視）でも数える（資産管理）でもない。

第4章

# 構成管理・変更管理

構成品目とベースライン ／ 変更管理の手順 ／ 構成状況の記録と完全性の保証 ／ リリース管理 ／ バージョン管理 ／ SBOM



# 構成管理：何がどの版の組合せでできているかを、いつでも言えるようにする

**構成識別**
品目を決め、名前と版番号
→
**変更管理**
承認した変更だけ入れる

**構成状況の記録**
状況と履歴を文書に
→
**完全性の保証**
機能的・物理的
→
**リリース・出荷**
版を出して保管

| 用語 | 中身 |
|---|---|
| 構成品目 | ソースコード・実行プログラム・設計書・テスト仕様書・**開発ツール**など |
| 構成管理計画 | 何を品目にし、誰がどう承認するかを最初に決める |
| 構成状況の記録 | 変更の回数・最新の版・移行の状況 |
| 保管期間 | 出荷した版のコードと文書は、ソフトウェアの寿命の間保管 |

**ベースライン：承認して凍結した基準点**




ここから先は変更管理の手続でしか変えられない

| | 機能的な完全性 | 物理的な完全性 |
|---|---|---|
| 確かめること | 要件・仕様の機能と性能を**全部満たす** | 記録どおりの品目と版が**全部そろう** |
| 手段 | テスト・レビューの結果 | 構成の記録と出荷物の照合 |

> 完全性の保証（構成監査）で一貫性と正確性を確かめてから出荷する。

# 変更管理の順番：影響を調べてから承認し、承認してから変える




| 段階 | 中身 |
|---|---|
| 変更要求 | 内容・理由・緊急度を書いた正式な申し出 |
| 影響分析 | ほかのプログラム・文書・テスト・日程・費用への影響を調べる |
| 変更管理委員会 | 変更を承認するかを決める会議体（CCB） |
| 変更の実施 | 承認された変更だけを入れ、回帰テストで確かめる |
| 構成状況の記録 | 新しい版と変更の履歴を記録 |

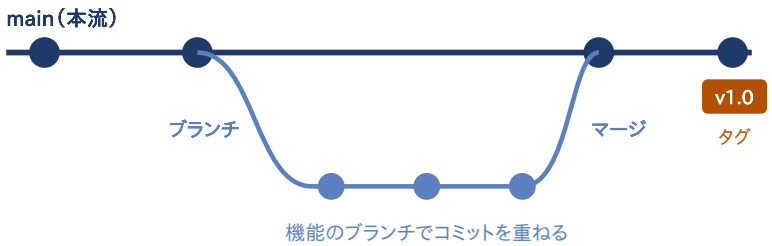
誤りの選択肢の型：「先に変更してから承認」「影響を調べずに承認」。**影響分析 → 承認 → 実施**の順は崩さない。

SBOM（ソフトウェア部品表）の例

| 部品 | 版 | ライセンス |
|---|---|---|
| OpenSSL | 3.0.13 | Apache-2.0 |
| zlib | 1.3.1 | Zlib |
| jQuery | 3.7.1 | MIT |

部品に**脆弱性**が公表されたとき、影響を受ける製品と版をすぐに特定できる。OSSのライセンスの条件を守る管理にも使う。

# バージョン管理：枝分かれして作業し、まとめて本流に戻す




| 用語 | 中身 |
|---|---|
| リポジトリ | ファイルと変更履歴を保存する場所 |
| コミット | 変更を説明付きで記録する |
| コンフリクト | 同じ所を別々に変えていて自動でまとめられない |
| プルリクエスト | 本流への取込みを依頼し、レビューしてもらう |
| チェックアウト | 作業する版を取り出す |

| | 集中型（Subversion） | 分散型（Git） |
|---|---|---|
| リポジトリ | サーバに**一つだけ** | 各自の手元に**完全な複製** |
| 記録 | サーバにつながっていないとできない | 手元でできる。後でやり取り |
| 同時の変更 | ロックして排他にする方式もある | ブランチとマージで並行して進める |

**版数管理**：版番号に規則を決める（例 2.1.3＝大きな変更.機能追加.修正）。番号で変更の大きさと新旧が分かる。

タグは付けた位置から動かない目印、ブランチは作業とともに先へ進む枝。

# まとめ：この分野で迷いやすい組と、見分ける一言

| 迷いやすい組 | 見分ける一言 | 迷いやすい組 | 見分ける一言 |
|---|---|---|---|
| 段階的 ／ 進展的モデル | 要件は最初に確定 ／ 要件も途中で見直す | 継続的デリバリー ／ デプロイ | 本番は人が判断 ／ 本番まで自動 |
| XP ／ スクラム | 開発のやり方 ／ チームの枠組み | Four Keys | 頻度・リードタイム＝速さ、障害率・修復時間＝安定性 |
| PO ／ SM ／ 開発者 | 優先順位とリリース判断 ／ 支援 ／ 作り方と作成 | ローコード ／ ノーコード | 一部だけ書く ／ 一切書かない |
| レビュー ／ レトロスペクティブ | 作ったものを見せる ／ 進め方をふりかえる。この順 | 形式手法 ／ 構造化手法 | 厳密な言語で検証 ／ 上から分ける |
| ベロシティ ／ スプリント | 生産量の尺度 ／ 期間の区切り | 能力水準 3 ／ 4 | 組織の標準プロセス ／ 定量的に管理 |
| バーンダウン ／ バグ曲線 | 残りが右下がり ／ 累積バグがS字 | JIS X 33002 ／ 0160 | アセスメントの実施（診断） ／ SLCPの共通枠組み |
| インセプションデッキ ／ プランニングポーカー | 決まった問いで目的をそろえる ／ 一斉に出して見積り | 著作者人格権 ／ 著作財産権 | 譲れない（行使しないと約束） ／ 譲れる |
| Keep ／ Problem ／ Try | 続ける ／ 困った事実 ／ 次から試す行動 | 専用 ／ 通常実施権 | 独占・登録・権利者も実施不可 ／ 独占でない |
| TDD ／ ペアプログラミング | 先にテストコード ／ 2人で1つのプログラム | IDE ／ ICE | 道具をまとめたソフト ／ CPUの情報を取る装置 |